

# PAYONE Composable Storefront — User Guide

Reusable PAYONE **Hosted Tokenization** integration for SAP Commerce Cloud Composable Storefront.

Published as `@payone/composable-storefront` .

## Scope

The library owns:

- the Hosted Tokenization SDK and iframe lifecycle;
- UI, locale, masking, cardholder-name, validation-icon, and Click to Pay configuration;
- normalized tokenization callbacks;
- `/authentication-token` , `/commerce-case` , and `/placeorder` calls through the SAP Commerce backend plugin;
- full-page navigation for `REDIRECTED` place-order responses;
- the fixed card-scheme-to-product-ID mapping: Visa ( `1` ), American Express ( `2` ), Mastercard ( `3` ), Diners Club ( `132` ).

The library only transports the storefront-authored `/placeorder` request and response; it does not define those DTOs. The consuming storefront owns those DTOs, checkout navigation, non-redirect result handling, order confirmation, SEPA and any other payment methods. **The browser never calls PAYONE REST APIs directly**, and the library never handles raw PAN or CVV — those live only inside the PAYONE-hosted iframe.

Requires Angular 21.2, Spartacus (Composable Storefront) 221121.15.1, RxJS 7.8, and `pcp-client-javascript-sdk` 1.4.0. SSR/hydration safe.

## 1. Build the library

From the workspace root:

```
npm install
npm run lint
npm test
npm run build          # ng build composable-storefront -> dist/composable-storefront
```

Package it for consumption by another project:

```
npm run pack:dry-run    # inspect the tarball contents first
npm run pack            # builds, then npm pack ./dist/composable-storefront
```

This produces a versioned tarball, e.g. `payone-composable-storefront-0.1.1.tgz` , in the workspace root.

## 2. Install into a Spartacus storefront

**Published registry:**

```
npm install @payone/composable-storefront
```

**Local development (no registry yet):** copy the tarball built in step 1 into the consuming storefront's `local-packages` directory and install the exact version from disk:

```
npm install ./local-packages/payone-composable-storefront-0.1.1.tgz
```

Rebuild and re-copy the tarball after every library change; there is no `npm link` watch flow for this package.

### 3. Register the provider

In the storefront's standalone bootstrap config (or a feature module's providers array), register Hosted Tokenization:

```
import { providePayoneHostedTokenization } from '@payone/composable-storefront';

providePayoneHostedTokenization({
  iframe: { height: 'auto', width: 400, zIndex: 9999 },
  isSecurityCodeMasked: false,
  showCardholderName: true,
  strictCardholderName: true,
});
```

This registers, scoped to Hosted Tokenization only:

- the Hosted Tokenization SDK wrapper ( `PayoneTokenizer` );
- `PayoneBackendPluginService` and its OCC adapter/connector;
- `PayonePaymentRedirectService` ;
- OCC endpoint configuration for `authentication-token` , `commerce-case` , and `placeorder` **only** — no other endpoints are added.

All options are optional and merge over library defaults. No option accepts a secret, amount, currency, payment product ID, or authorization mode — those remain backend-authored.

### 4. Create the session

Call `PayoneBackendPluginService` (backed by the SAP Commerce backend plugin) to obtain the pieces needed for a session:

```
import { inject } from '@angular/core';
import { PayoneBackendPluginService } from '@payone/composable-storefront';

const payoneBackend = inject(PayoneBackendPluginService);

payoneBackend.createAuthenticationToken(); // -> { token, id?, expirationDate }
payoneBackend.createCommerceCase();      // -> { commerceCaseId, checkoutId,
amount, currencyIsoCode, ... }
```

Both calls resolve the active user and cart automatically and require a logged-in user with an active cart. Build a `PayoneHostedTokenizationSession` from the returned token plus backend-controlled `mode` , `locale` , and `allowedCardSchemes` .

## 5. Render the component

```
<payone-hosted-tokenization
  [session]="cardSession"
  submitLabel="Continue with card"
  submitButtonClass="btn btn-primary btn-block"
  (ready)="onCardReady()"
  (tokenized)="onCardTokenized($event)"
  (tokenizationFailed)="onCardTokenizationFailed($event)"
/>
```

| I/O                         | Type                            | Notes                                                         |
|-----------------------------|---------------------------------|---------------------------------------------------------------|
| session (input, required)   | PayoneHostedTokenizationSession | rebuilds the iframe whenever it changes                       |
| submitLabel (input)         | string                          | default 'Submit'                                              |
| submitButtonClass (input)   | string                          | CSS classes for the submit button                             |
| ready (output)              | void                            | secure fields initialized                                     |
| tokenized (output)          | PayoneTokenizerSuccess          | contains paymentProcessingToken, cardType, masked cardDetails |
| tokenizationFailed (output) | PayoneTokenizerFailure          | recoverable — the submit button stays available               |

The component cleans up its own DOM resources on destroy ( `ngOnDestroy` ); the underlying SDK has no `public destroy / submit` method to call.

## 6. Place the order

On `tokenized`, store the opaque `paymentProcessingToken` and `cardType` until the storefront's own Review Order / place-order step, then map the scheme to the fixed PAYONE product ID:

```
import {
  getPayonePaymentProductId,
  PayoneBackendPluginService,
  PayonePaymentRedirectService,
} from '@payone/composable-storefront';

const paymentProductId = getPayonePaymentProductId(result.cardType);
// Visa -> 1, American Express -> 2, Mastercard -> 3, Diners Club -> 132
```

The storefront builds its own place-order request DTO — the library defines neither the request nor the result type — and passes it to `PayoneBackendPluginService.placeOrder()`, which posts it to the backend plugin's `/placeorder` endpoint:

```
const payoneBackend = inject(PayoneBackendPluginService);

payoneBackend.placeOrder<MyPlaceOrderResult>(myPlaceOrderRequest)
  .subscribe((result) => { /* storefront-defined result shape */ });
```

Use an `Idempotency-Key` header on order placement so retries are safe. Only `PaymentExecution SALE` is supported; `PRE_AUTHORIZATION` and `OrderManagementCheckoutActions` are out of scope for this library.

If the response indicates a `REDIRECTED` payment status, pass the status and redirect URL to `PayonePaymentRedirectService.redirectIfRequired()`. It performs the SSR-safe full-page navigation and returns `true` when it handled the result as a redirect, so the storefront can skip its own order-confirmation handling for that case:

```
const redirectService = inject(PayonePaymentRedirectService);

const handled = redirectService.redirectIfRequired(
  result.paymentStatus,
  result.redirectUrl,
);
```

## Reference

- [docs/storefront-integration.md](#) — worked example against the supplied electronicsstore
- [docs/api-contract.md](#) — backend-plugin endpoint contract
- [docs/architecture.md](#) — component/service architecture

Never commit npm registry credentials, PAYONE credentials, authentication tokens, or payment processing tokens.